

---

Oliver Krauss

# Maven, Encapsulation & JavaFX Intro

ALD | MTD.ba VZ SS26

Hagenberg 12.03.2026

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY  
OF APPLIED SCIENCES  
UPPER AUSTRIA

# Recap: Where We Left Off

Last session you learned **Object-Oriented Programming** fundamentals:

- Classes bundle data (fields) and behavior (methods)
- Constructors initialize objects via **new**
- **this** refers to the current object
- Objects are references; arrays of objects
- JUnit tests with **@Test** and **assertEquals**

**Today:** We need dependency management (Maven), data protection (encapsulation), and visual output (JavaFX).

# The Problem

Last session, you used JUnit -- but where did it come from?

- JUnit is an **external library** -- it does not ship with Java
- JavaFX is also external since Java 11
- How do you add external libraries to your project?
- How do you make sure everyone on your team uses the **same version**?
- Imagine 5 team members, each with a different JUnit version -- tests pass for you, fail for them

# The Problem

Last session, you used JUnit -- but where did it come from?

- JUnit is an **external library** -- it does not ship with Java
- JavaFX is also external since Java 11
- How do you add external libraries to your project?
- How do you make sure everyone on your team uses the **same version**?
- Imagine 5 team members, each with a different JUnit version -- tests pass for you, fail for them

## The manual approach (don't do this)

Download **.jar** files from the internet, put them in a **lib/** folder, add them to the classpath manually. Breaks when versions change, impossible to share reliably.

We need a **build tool** that handles this for us.

# What is Maven?

**Maven** is a build tool and dependency manager for Java:

- **Dependency management** -- declare what libraries you need, Maven downloads them
- **Build automation** -- compile, test, and package with one command
- **Project structure** -- enforces a standard directory layout
- **Reproducibility** -- everyone gets the same dependencies, same versions

## Analogy

Maven is like a **cook** for your project. You list the ingredients (dependencies) and the cooking steps (build lifecycle). Maven fetches the ingredients and follows the steps automatically.

*Note: Gradle is another popular build tool. We use Maven because it is simpler to learn and more widely used.*

# Maven Project Structure

Maven enforces a **standard directory layout**:

```
my-project/  
├── pom.xml  
└── src/  
    ├── main/  
    │   └── java/  
    │       └── your classes here  
    └── test/  
        └── java/  
            └── your tests here
```

- `pom.xml` -- the project configuration file
- `src/main/java/` -- your production code (`Circle.java`, `Player.java`)
- `src/test/java/` -- your test code (`CircleTest.java`)

## Important

This structure is **not optional**. Maven will not find your code if it is in the wrong directory. Do not put files “wherever” -- follow the layout exactly.

# The POM File

`pom.xml` (Project Object Model) is Maven's configuration file:

```
1  <?xml version="1.0" encoding="UTF-8
    ">
2  <project>
3      <modelVersion>4.0.0</
        modelVersion>
4
5      <groupId>at.fhooe.ald</groupId>
6      <artifactId>my-project</
        artifactId>
7      <version>1.0</version>
8
9      <dependencies>
10         <!-- libraries go here -->
11     </dependencies>
12 </project>
```

- **groupId** -- who made it (like a package name, typically reversed domain)
- **artifactId** -- the project name
- **version** -- the version number
- These three together are called **Maven coordinates** -- they uniquely identify any project or library
- Think of it like a postal address: groupId = city, artifactId = street, version = house number

# Adding Dependencies: JavaFX and JUnit

You declare what you need using **Maven coordinates** in the `<dependencies>` section. Maven uses these to fetch the libraries.

```
1    <dependencies>
2        <dependency>
3            <groupId>org.openjfx</groupId>
4            <artifactId>javafx-controls</artifactId>
5            <version>25.0.2</version>
6        </dependency>
7        <dependency>
8            <groupId>org.junit.jupiter</groupId>
9            <artifactId>junit-jupiter</artifactId>
10           <version>5.10.2</version>
11           <scope>test</scope>
12        </dependency>
13    </dependencies>
```

Here we add **two** dependencies: **JavaFX** (for the application) and **JUnit** (for tests). The `<scope>` on JUnit means something special...



# What Does `<scope>` Do?

**Scope** controls where a dependency is available:

- **No scope** (or **compile**) -- available in main code *and* test code. Use this for libraries your application needs at runtime (e.g. JavaFX).
- **test** -- only available in test code. Use this for testing libraries (e.g. JUnit). They are not packaged into your final application.

So: JavaFX has no `<scope>` because we need it to run the app. JUnit has `<scope>test</scope>` because we only need it when running tests.

# Dependency Scopes at a Glance

| Scope                    | Available when         | Packaged?                    |
|--------------------------|------------------------|------------------------------|
| <b>compile</b> (default) | compile & run          | yes                          |
| <b>test</b>              | test only              | no                           |
| <b>runtime</b>           | run only (not compile) | yes                          |
| <b>provided</b>          | compile & test         | no (provided by environment) |

**Examples:** **compile** -- JavaFX; **test** -- JUnit; **runtime** -- JDBC driver; **provided** -- Servlet API.

# Maven Downloads Everything a Library Needs

When you add `javafx-controls`, Maven doesn't just download that one JAR. JavaFX itself depends on other modules (`javafx-graphics`, `javafx-base`, etc.). Maven follows these **transitive dependencies** and downloads them automatically.

## Why this matters

Without a tool like Maven, you'd have to find, download, and configure every JAR by hand -- and if library A needs B and B needs C, and so on.

You quickly hit **dependency hell**. Maven resolves the whole tree for you.

# Where Does Maven Download From?

## Maven Central

**search.maven.org** -- a public repository with thousands of Java libraries.

- Search for a library (e.g. JUnit, JavaFX), copy its **coordinates** (**groupId**, **artifactId**, **version**) into your **pom.xml**
- Maven downloads the library and any transitive dependencies automatically
- There are other repositories, but Maven Central is the most common.
- You can set up your own repository if you want to keep libraries private (Nexus).

# Extending Maven with Plugins

## `<dependencies>`

Libraries your **code uses**:

- JavaFX controls
- JUnit
- Logging libraries

These become part of your **application**.

## `<build><plugins>`

Tools that change **how Maven builds**:

- Configure the Java compiler
- Add new **mvn** goals
- Post-process output

These extend **Maven itself**.

### **Two plugins we use in this course:**

- **maven-compiler-plugin** --- tells **javac** to use **Java 25**
- **javafx-maven-plugin** --- adds the **mvn javafx:run** goal and sets up the JavaFX module path at runtime

Plugins use the same coordinates as dependencies (**groupId + artifactId + version**) and are also downloaded from Maven Central.

# The <build> Section in Our pom.xml

```
1  <build>
2    <plugins>
3      <plugin>
4        <groupId>org.apache.maven.plugins</groupId>
5        <artifactId>maven-compiler-plugin</artifactId>
6        <version>3.11.0</version>
7        <configuration>
8          <release>25</release>      <!-- Java 25 -->
9        </configuration>
10     </plugin>
11     <plugin>
12       <groupId>org.openjfx</groupId>
13       <artifactId>javafx-maven-plugin</artifactId>
14       <version>0.0.8</version>
15       <configuration>
16         <mainClass>at.fhooe.ald.shapes.ShapeApp</
17         mainClass>
18       </configuration>
19     </plugin>
20   </plugins>
21 </build>
```

## maven-compiler-plugin

Without it, older Maven defaults may not handle **module-info.java** correctly. Setting **<release>25</release>** ensures **javac** targets Java 25 with full Java Platform Module System (JPMS) support.

## javafx-maven-plugin

Without it, **mvn javafx:run** does not exist. It adds the goal and sets up **--module-path** so JavaFX native libraries are found at runtime.

# What Can the Build System Be Used For?

Plugins in `<build>` extend Maven with automation beyond compile and run. Examples:

- **Testing & quality** -- JUnit, TestNG; code coverage (JaCoCo, Cobertura); static analysis (Checkstyle, SpotBugs)
- **Packaging & deployment** -- JAR/WAR creation, assembly of distributions, deploy to repository or server (e.g. Tomcat, Maven Central)
- **Documentation** -- Javadoc, project site, dependency reports
- **Code generation** -- annotation processors, WSDL/Schema codegen, protobuf
- **Native / desktop** -- JavaFX (as in our project), GraalVM native image, installers (jpackage, launch4j)

One command (`mvn test`, `mvn package`, ...) runs the whole chain; no need to remember compiler flags or classpaths.

# Maven Build Lifecycle

Maven has three built-in **lifecycles**: **default** (build & deploy), **clean** (remove build output), **site** (generate documentation). Each lifecycle is a sequence of **phases** -- there are about 30 phases in total; we focus on the most important.



## Key phases (default lifecycle):

- `mvn compile` -- compiles `src/main/java`
- `mvn test` -- runs all previous phases **and** JUnit tests
- `mvn package` -- runs previous phases **and** creates the artifact
- Running a phase runs all earlier phases in that lifecycle

In IntelliJ: **Maven** tool window (right) or green **Run**.



# What Is Packaging?

**Packaging** is the phase that turns your compiled code into a distributable **artifact** -- usually a **.jar** file (Java ARchive).

- The JAR contains your compiled **.class** files plus resources (images, config files). It is just a ZIP file with a standard layout.
- `<packaging>jar</packaging>` in the POM (or omitted; **jar** is the default) means **mvn package** produces a **.jar** in **target/**.
- Other packaging types exist (e.g. **war** for web apps, **pom** for parent POMs); for our course we use **jar**.

So: **mvn package** = compile + test + *package everything into a single JAR you could share or run.*

# Quick Check: What Goes Where?

**Match each file to its correct Maven directory:**

**Files:**

1. Circle.java
2. CircleTest.java
3. Player.java
4. WeaponTest.java
5. pom.xml

**Directories:**

- A. src/main/java/
- B. src/test/java/
- C. Project root

# Quick Check: What Goes Where? - Answer

Where each file lives in the Maven layout:

```
project root/  
  pom.xml  
  src/  
    main/java/  
      Circle.java  
      Player.java  
    test/java/  
      CircleTest.java  
      WeaponTest.java
```

Production code in **main**, test code in **test**. **pom.xml** stays in the project root, not inside **src/**.

# Creating a Maven Project in IntelliJ

1. **File** → **New** → **Project**
2. Select **Maven Archetype** as the generator
3. Fill in **groupId**: `at.fhooe.ald` and **artifactId**: your project name
4. IntelliJ creates the standard directory structure and `pom.xml`
5. Open `pom.xml` and add your dependencies (JUnit, JavaFX)
6. IntelliJ shows a popup: **“Load Maven Changes”** -- click it

## Important

Always click **“Load Maven Changes”** after editing `pom.xml`. Otherwise IntelliJ does not know about your new dependencies and imports will show errors.

# Maven Summary

| Concept                         | What it does                   |
|---------------------------------|--------------------------------|
| <code>pom.xml</code>            | Project configuration file     |
| Maven coordinates               | groupId + artifactId + version |
| <code>&lt;dependency&gt;</code> | Declares an external library   |
| <code>src/main/java</code>      | Production code                |
| <code>src/test/java</code>      | Test code                      |
| <code>mvn compile</code>        | Compile source code            |
| <code>mvn test</code>           | Compile + run tests            |

**Key takeaway:** Maven lets you declare dependencies in XML and handles downloading, versioning, and building automatically. From now on, all our projects use Maven.

# Most Important `mvn` Commands

- `mvn clean install` -- clean `target/`, compile, run tests, package, and install the artifact into your local Maven repository. The command you use for a **full build**.
- `mvn clean install -DskipTests` -- same as above but **skip tests**. Faster when you only changed non-test code; use with care.
- `mvn javafx:run` -- run a JavaFX application. Requires the `javafx-maven-plugin` in your POM (we add it in the exercises).
- `mvn clean` -- delete `target/`. Useful when something is stuck or you want a fresh build.
- `mvn test` -- compile and run tests only (no package). Handy during test-driven development.

# Packages -- What They Are

A **package** is Java's way of grouping classes into a **namespace**:

- Avoids name clashes: **util.Date** vs **sql.Date** are different classes
- Convention: reverse domain name, e.g. **at.fhooe.ald.shapes** for our course
- Every class belongs to exactly one package (or the "default package" if you omit it; we do not use that)

In our projects we use a package like **at.fhooe.ald.shapes** so that our **Circle**, **Rectangle**, etc., live in a clear, predictable place. Other code (and Maven dependencies like JavaFX) also use packages -- which is why we need **imports** next.

# Packages -- Directory Structure

The folder structure under `src/main/java/` must **mirror** the package name:

`Package at.fhooe.ald.shapes`

`⇒ folder src/main/java/at/fhooe/ald/shapes/`

`Circle.java, Rectangle.java, ...`

- One dot in the package name = one directory level
- Put **Circle.java** in that folder and start the file with `package at.fhooe.ald.shapes;`
- Maven and the Java compiler expect this layout -- do not put classes in random folders



# Imports -- What For?

Once we have **packages** (our own and those from Maven dependencies), we need a way to use classes from other packages. That is what **imports** are for:

- Without import: you must use the **fully qualified class name** every time (e.g. `javafx.scene.canvas.Canvas`)
- With import: you write `import javafx.scene.canvas.Canvas;` once at the top of the file, then use `Canvas` in your code
- This is what we can do with Maven: add a dependency in the POM, then **import** and use its classes in our Java files

IntelliJ can add imports automatically when you use a class (e.g. Alt+Enter or “Add import”).

# Imports -- Example

```
1  package at.fhooe.ald.shapes;
2
3  import javafx.scene.canvas.Canvas; // from Maven dependency
4
5  public class MyApp {
6      public void draw() {
7          Canvas canvas = new Canvas(400, 300); // short name
8          // ...
9      }
10 }
```

Without the import line you would have to write `javafx.scene.canvas.Canvas canvas = ...` every time. One import per class (or `import javafx.scene.canvas.*;` for all classes in that package -- use sparingly).

# Where Do Modules Fit?

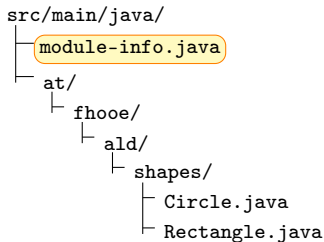
Java has two levels above classes:

1. **Package** -- groups classes (namespace). We just saw: `at.fhooe.ald.shapes`, directory structure, imports.
2. **Module** -- groups packages and declares what the module **requires** (other modules) and **exports** (which of its packages are visible to others).

JavaFX is distributed as modules, so we need a module descriptor. We will see the `module-info.java` file in detail right after this slide.

# module-info.java --- Where Does It Live?

The file has a fixed location: **directly** inside `src/main/java/` -- **not** inside any package folder.



- `module-info.java` sits at the **top level** of `src/main/java/`
- It is **not** inside `at/fhooe/ald/shapes/` -- that would be wrong
- One file per module (one module per Maven project)
- Package folders are **siblings**, not parents

## Common mistake

Placing `module-info.java` inside the package folder. The compiler will not find it.

# module-info.java --- Line by Line

```
1  module at.fhooe.ald.shapes {  
2      requires javafx.controls;  
3      exports at.fhooe.ald.shapes;  
4  }
```

- **module at.fhooe.ald.shapes** -- module name; convention: **groupId.artifactId**, matches your package
- **requires javafx.controls;** -- “I need this JavaFX module”; one **requires** per external module
- **exports at.fhooe.ald.shapes;** -- “my package is visible”; without this, nothing outside can use your classes

## For this course

Create once when you set up the project. Only add a line if you add a new JavaFX dependency (e.g., **requires javafx.fxml;**). Otherwise leave it alone.

# What Happens Without module-info.java?

## Without (or wrong)

- **javafx.controls** not found
- Red underlines in IntelliJ
- **IllegalAccessError** at run
- *"module javafx.graphics does not export javafx.scene to unnamed module"*

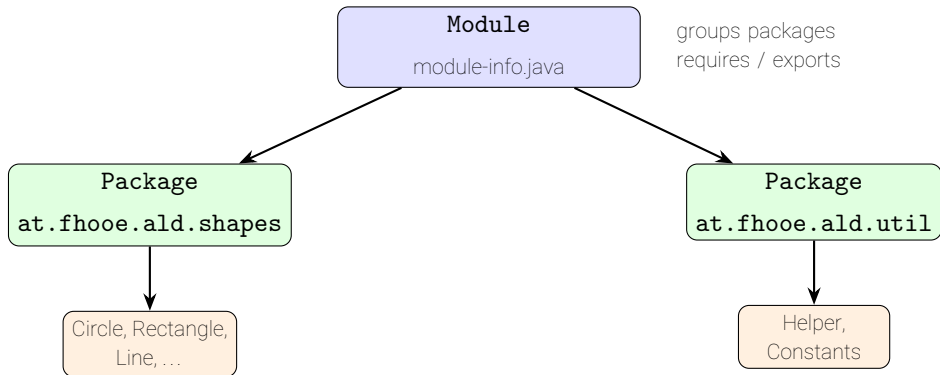
## With correct module-info.java

- JavaFX imports resolve cleanly
- **mvn javafx:run** works
- Window opens as expected

## Diagnosis tip

JavaFX classes **red in IntelliJ** but **pom.xml** has the dependency? Likely cause: missing or misplaced **module-info.java**. Check: **src/main/java/module-info.java**.

# Summary: Module, Package, Class



**One module** → **many packages** → **many classes** (each package has its own). In our course we often use one package per project for simplicity; real projects may have several. Imports let us use classes from other packages (e.g. JavaFX) inside our classes.

# The Problem: Unprotected Fields

Remember our **Circle** class from last session?

```
1  Circle c = new Circle(5.0);  
2  c.radius = -5.0;  // nobody stops us!  
3  IO.println(c.getArea()); // negative area??
```

- Anyone can set **radius** to **any value** -- even negative or zero
- The class has no way to **enforce rules** about valid data
- A **Circle** with negative radius makes no sense



# The Problem: Unprotected Fields

Remember our **Circle** class from last session?

```
1  Circle c = new Circle(5.0);  
2  c.radius = -5.0;  // nobody stops us!  
3  IO.println(c.getArea()); // negative area??
```

- Anyone can set **radius** to **any value** -- even negative or zero
- The class has no way to **enforce rules** about valid data
- A **Circle** with negative radius makes no sense

## More dangerous examples

`player.hp = 999999;` -- invincible player

`bankAccount.balance = 1000000;` -- instant millionaire

`weapon.durability = -1;` -- undefined behavior

# What is Encapsulation?

**Encapsulation** means hiding internal data and controlling access through methods:

## Without encapsulation

- Fields are directly accessible
- Anyone can read **and write**
- No validation possible
- "Hope nobody sets radius to **-5**"

## With encapsulation

- Fields are hidden (**private**)
- Access only through methods
- Methods can **validate** input
- "Radius is always **> 0**, guaranteed"

## Analogy

A bank does not let you walk into the vault and take money. You use the **counter** (methods) -- the teller checks your identity and balance before handing out cash.

# Access Modifiers

Java provides keywords to control **who can access** a member. You can use them on **fields**, **methods**, **constructors**, and (for visibility) **classes**.

| Modifier               | Who can access?           | Use for                                  |
|------------------------|---------------------------|------------------------------------------|
| <code>public</code>    | Everyone, from anywhere   | API methods, constructors                |
| <code>private</code>   | Only the class itself     | Fields (almost always), internal helpers |
| <code>protected</code> | Same package + subclasses | Members subclasses may use               |
| (none) package-private | Same package only         | Package-internal helpers                 |

**Golden rule for now:** Fields `private`; methods `public` (if others need them) or `private` (internal only). We use `protected` and package-private when we do inheritance.

## Common misconception

`private` does not mean “secret” or “encrypted.” It is an **access rule enforced by the compiler** -- other classes cannot touch the field.

# Where the Modifiers Go

```
1  public class Circle {  
2      private double radius;  
3      public Circle(double r) {  
4          this.radius = r;  
5      }  
6      public double getArea() {  
7          return Math.PI * radius *  
8              radius;  
9      }  
}
```

**Class:** `public` = visible to everyone

**Field:** `private` = only this class

**Constructor:** `public` so callers can `new` `Circle(...)`

**Method:** `public` = part of the API

# Making Fields Private

Adding **private** prevents direct access from outside the class:

```
1  class Circle {
2      private double radius; // only Circle can access
3
4      Circle(double radius) {
5          this.radius = radius;
6      }
7
8      double getArea() {
9          return Math.PI * this.radius * this.radius;
10     }
11 }
```

# Getters: Controlled Read Access

A **getter** is a method that returns the value of a private field:

```
1  class Circle {  
2      private double radius;  
3  
4      Circle(double radius) {  
5          this.radius = radius;  
6      }  
7  
8      public double getRadius() {  
9          return this.radius;  
10     }  
11 }
```

# Setters: Controlled Write & Validation

A **setter** is a method that changes a private field -- with validation:

```
1  class Circle {
2      private double radius;
3
4      public void setRadius(double radius) {
5          if (radius <= 0) {
6              IO.println("Error: radius must be positive!");
7              return;
8          }
9          this.radius = radius;
10     }
11 }
```

# Getter/Setter Summary

- **Getter** (`getX()`): returns the value of a private field. Read-only access for callers.
- **Setter** (`setX(value)`): updates the field, ideally with validation. Controlled write access.
- **Naming**: `get` + FieldName (or `is` + FieldName for `boolean`).
- Not every field needs both -- expose only what other classes need. Immutable data often has getters but no setters.

## Recap

Private fields + public getters/setters where needed = encapsulation. The class stays in control of its data.



# Java Beans (Convention)

**JavaBeans** is a naming and structure convention used by many frameworks (GUI builders, JSON libraries, etc.):

- A “bean” has a no-argument constructor, and for each “property” either a **getX()** / **setX()** pair or **isX()** for booleans.
- Optionally implements **Serializable**. The name comes from Java’s original JavaBeans API.
- Many tools expect this pattern (e.g. reflection-based mapping). You will see it in real projects.

For this course we are not strict about the full bean convention (e.g. we use constructors with parameters). Knowing the term and the **get\*/set\*** naming helps when you meet it elsewhere.

# Fluent Interface: Setters That Return **this**

A **fluent** style lets callers chain method calls by having setters return **this**:

```
1  public Circle setRadius(double radius) {  
2      if (radius > 0) this.radius = radius;  
3      return this; // allow chaining  
4  }
```

```
1  Circle c = new Circle(1.0);  
2  c.setRadius(5.0).setX(10).setY(20); // chained calls
```

This is a **style choice**, not required for encapsulation. Useful for builders and configuration objects. Standard getters/setters (returning **void**) are still the most common.

# Side by Side: Before and After Encapsulation

## Before

```
1  class Circle {
2      double radius;
3
4      Circle(double radius) {
5          this.radius = radius;
6      }
7  }
8
9  // anyone can break it
10 Circle c = new Circle(5.0);
11 c.radius = -5.0; // no error!
```

## After (encapsulated)

```
1  class Circle {
2      private double radius;
3      Circle(double radius) {
4          this.radius = radius;
5          if (radius <= 0) {
6              this.radius = 1.0;
7          }
8      }
9      public double getRadius() {
10         return this.radius;
11     }
12     public void setRadius(double radius) {
13         if (radius > 0) {
14             this.radius = radius;
15         }
16     }
17 }
```

# Encapsulated Player Class

```
1  class Player {
2      private String name;
3      private int hp;
4      private int maxHp;
5      private int damage;
6
7      Player(String name, int hp, int damage) {
8          this.name = name;
9          this.maxHp = hp;
10         this.hp = hp;
11         this.damage = damage;
12     }
13
14     public String getName() { return this.name; }
15     public int getHp() { return this.hp; }
16     public int getDamage() { return this.damage; }
17     public void takeDamage(int amount) { this.hp = Math.max(0, this.hp - amount); }
18     public void heal(int amount) { this.hp = Math.min(this.maxHp, this.hp + amount); }
19 }
```

Page 41 | 82

# Why Not Just a Setter for HP?

## Setter approach

```
1  public void setHp(int hp) {
2      this.hp = Math.max(0,
3          Math.min(this.maxHp, hp));
4  }
5
6  // caller must compute
7  player.setHp(
8      player.getHp() - 25);
```

## Behavior approach

```
1  public void takeDamage(
2      int amount) {
3      this.hp = Math.max(0,
4          this.hp - amount);
5  }
6
7  // caller just says what
8  player.takeDamage(25);
```

## Principle

Prefer **meaningful methods** (`takeDamage`, `heal`) over raw setters (`setHp`). The setter forces the caller to know the implementation. The behavior method lets the object manage itself. This is the heart of OOP.

# toString() with @Override

In Session 1 we wrote `toString()` without `@Override`. Now we do it properly:

```
1  class Circle {  
2      private double radius;  
3      // constructor omitted  
4  
5      @Override  
6      public String toString() {  
7          return "Circle(radius=" + this.radius + ")";  
8      }  
9  }
```

# @Override and the Contract

- **@Override** tells the compiler: “I am replacing a method from the parent class”
- Every class inherits from **Object**, which has a default **toString()**
- With **@Override**, a misspelled method name causes a **compile error** instead of a silent bug
- Without **@Override**: your **toString()** still works, but the compiler cannot check that you really meant to override **Object**’s method (typos go unnoticed).
- With **@Override**: you are declaring a contract -- “**Object** has **toString()**, and I provide my version.” The compiler enforces that a matching method exists in the parent.

## Rule

Always use **@Override** when writing **toString()**. We will see *why* this works (inheritance) later.

# The `equals()` Problem: Why `==` Fails

Remember from Session 1: `==` on objects compares **references**, not values:

```
1  Circle a = new Circle(5.0);
2  Circle b = new Circle(5.0);
3  IO.println(a == b); // false! Different objects in memory
```

Two circles with the same radius -- but `==` says they are different!

We need a way to compare by **field values**. Java's answer: the `equals()` method.

```
1  IO.println(a.equals(b)); // true! Same radius
```

But how do we implement `equals()`? Let's build it step by step.



## equals() -- Same Idea as toString()

Like `toString()`, `equals()` comes from `Object`. The default implementation compares **references** (same object in memory). We `@Override` it to compare by **content** (field values).

```
1  @Override
2  public boolean equals(Object obj) {
3      if (this == obj) return true;           // 1. same object?
4      if (!(obj instanceof Circle)) return false; // 2. correct type?
5      Circle other = (Circle) obj;           // 3. cast to Circle
6      return this.radius == other.radius;    // 4. compare fields
7  }
```

1. **Same reference?** `a.equals(a) ⇒ true`
2. **Type check:** `instanceof Circle` (and `null` is not an instance)
3. **Cast:** `Object → Circle` to access `radius`
4. **Compare fields:** `same radius ⇒ equal circles`

# `equals()` with Multiple Fields & Tools

- **You decide** which fields define equality. For **Rectangle**, usually **width** and **height**; for **Circle**, **radius**. Compare **all** fields that should matter: `return this.width == other.width && this.height == other.height;`
- **IntelliJ** can auto-generate `equals()` (and `hashCode()`, which we touch later): right-click in the class → Generate → `equals()` and `hashCode()`. Pick the fields to include. Then review the generated code so you understand the pattern.

*If you override `equals()`, you should override `hashCode()` too -- we will see why when we use `HashMap` and sets.*

# equals() -- Don't Panic

You do not need to memorize this pattern

You need to understand **why** `equals()` exists and **when** to use it.

IntelliJ can generate the implementation for you: **Code** → **Generate** → **equals()** and **hashCode()**.

## Key takeaways:

- `==` compares references ("same object in memory?")
- `equals()` compares values ("same field values?")
- Use `equals()` whenever you want to check if two objects are *logically* the same
- The parameter is **Object** because every class inherits from **Object** (more in Session 3)

Recipe (copy-paste template)

```
@Override → this == obj? → instanceof? → cast → compare fields
```

# Quick Check: Predict the Output

```
1  Circle a = new Circle(5.0);
2  Circle b = new Circle(5.0);
3  Circle c = a;
4
5  IO.println(a == b);      // ???
6  IO.println(a == c);      // ???
7  IO.println(a.equals(b)); // ???
8  IO.println(a.equals(c)); // ???
```

## Quick Check: Predict the Output - Answer

(One) Answer)

```
1  IO.println(a == b);          // false  -- different objects
2  IO.println(a == c);          // true   -- same reference
3  IO.println(a.equals(b));      // true   -- same field values
4  IO.println(a.equals(c));      // true   -- same object, same values
```

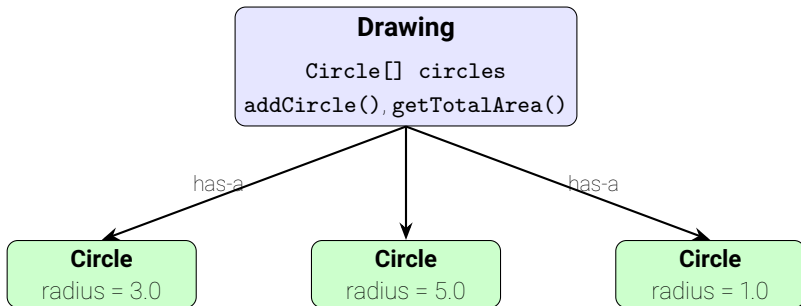
# Composition: Has-A Relationship

In Session 1 we saw **Player** having a field **Weapon**. This is **composition** (has-a):

```
class Drawing {
    private Circle[] circles;
    private int count;
    Drawing(int maxShapes) {
        this.circles = new Circle[maxShapes];
        this.count = 0;
    }
    public void addCircle(Circle c) {
        if (this.count < this.circles.length) {
            this.circles[this.count] = c;
            this.count++;
        }
    }
    public double getTotalArea() {
        double total = 0;
        for (int i = 0; i < this.count; i++)
            total += this.circles[i].getArea();
        return total;
    }
}
```

# Composition: The Big Picture

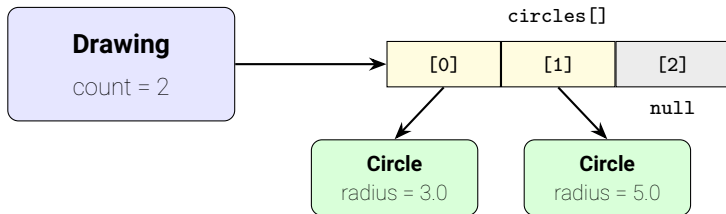
*Composition: "A Drawing **has** Circles"*



- Composition = an object contains other objects as fields
- The **Drawing has** circles; the circles do not know about the drawing

# Composition: Memory View

What does this look like in memory after `addCircle(c1); addCircle(c2);`?



*Circle objects live on the **heap**; the array holds **references** to them*



# Your Turn: Design Getters/Setters for Weapon

Exercise (3 minutes -- discuss with your neighbor)

The **Weapon** class from Session 1 has these fields:

```
-- String name; int damage; int durability;
```

Decide for each field:

1. Should it have a **getter**?
2. Should it have a **setter**? If so, what **validation**?
3. Or should it have a **behavior method** instead?

*Which field should NOT have a setter and why? Discuss with your neighbor.*

# Design Getters/Setters for Weapon - Answer

One possible answer

**name**: getter only (weapons don't change their name)

**damage**: getter only (or setter with **damage** > 0 check)

**durability**: getter + **no setter** -- use **use()** method that decreases it

# Spot the Bug!

**Each snippet has an encapsulation problem. Can you find it?**

Snippet 1

```
1  class Circle {  
2      private double radius;  
3  
4      public void setRadius(  
5          double radius) {  
6          this.radius = radius;  
7      }  
8  }
```

Snippet 2

```
1  class Player {  
2      private int hp;  
3      public int[] scores;  
4  
5      Player(int hp) {  
6          this.hp = hp;  
7          this.scores = new int[10];  
8      }  
9  }
```

# Spot the Bug! - Answer

## Answers

**Snippet 1:** Setter has **no validation** -- `setRadius(-5.0)` still works. Add an `if (radius > 0)` check.

**Snippet 2:** `scores` is **public** -- anyone can modify the array directly. All fields should be **private**.

# Encapsulation Recap

Before we move to JavaFX, let's make sure the key ideas are clear:

- Make fields **private** -- always
- Provide **public** getters for read access
- Provide setters **only when needed**, with validation
- Prefer behavior methods (**takeDamage**) over raw setters (**setHp**)
- **@Override toString()** for readable output
- **@Override equals()** for value-based comparison
- Composition: objects containing other objects (has-a)

*Any questions before we start drawing things on screen?*

# What is JavaFX?

**JavaFX** is Java's toolkit for creating graphical user interfaces (GUIs):

- Windows, buttons, text fields -- desktop applications
- **Canvas** -- a drawing surface where we can paint shapes
- We will use Canvas to **visualize our OOP classes**
- Circle objects from our code → circles on screen

## Why JavaFX in this course?

- Makes OOP **tangible** -- you can see your objects
- Motivates encapsulation -- shapes need valid positions and sizes
- Later: event handling, MVC pattern, interactive applications

# Why JavaFX for OOP Learning?

## The key insight

JavaFX turns your OOP code into something you can **see and interact with**. Your **Circle**, **Rectangle**, and **Line** classes draw themselves on screen -- that is the point.

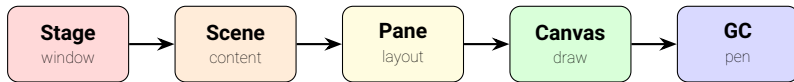
- **OOP becomes tangible**: write a **Circle** class, see a circle on screen -- instant feedback
- **Encapsulation has consequences**: invalid radius = broken shape on screen, not just a wrong number
- Honest context: most new projects use web or mobile UI (not Java desktop)
- But the concepts -- canvas, coordinates, event handling -- are **universal**

You are learning OOP, not “production Java UI”

The visual output is a tool for understanding objects. The same OOP principles apply whether you later work in web, mobile, or desktop.

# JavaFX Architecture

Every JavaFX application has a hierarchy of containers:



**Window** → **Page** → **Layout** → **Drawing board** → **Pen** (GraphicsContext)



# Your First JavaFX Application (minimal)

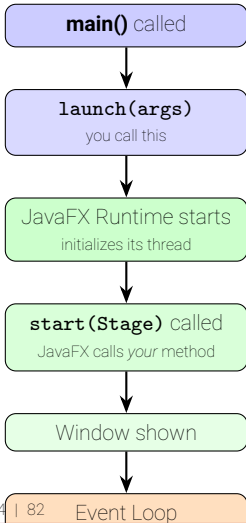
```
1  public class MyApp extends Application {
2      @Override
3      public void start(Stage stage) {
4          Canvas canvas = new Canvas(800, 600);
5          GraphicsContext gc = canvas.getGraphicsContext2D();
6          gc.fillRect(100, 100, 200, 150); // draw a rectangle
7
8          Pane pane = new Pane(canvas);
9          Scene scene = new Scene(pane);
10         stage.setTitle("My First JavaFX App");
11         stage.setScene(scene);
12         stage.show();
13     }
14
15     public static void main(String[] args) {
16         launch(args); // starts the JavaFX runtime
17     }
18 }
```

# Understanding the Code

Let's break down the key parts:

- **extends Application** -- treat this as a **given by JavaFX** for now; it means "this is a JavaFX app" (we explain **extends** in Session 3)
- **@Override public void start(Stage stage)** -- JavaFX calls this method when the app starts; the **Stage** is the window
- **new Canvas(800, 600)** -- creates an 800×600 pixel drawing surface
- **canvas.getGraphicsContext2D()** -- gets the "pen" for drawing
- **gc.fillRect(100, 100, 200, 150)** -- draws a filled rectangle at position (100, 100) with width 200 and height 150
- **launch(args)** in **main** -- initializes JavaFX and calls your **start()** method; the window stays open until the user closes it

# How `launch()` Works



What you write

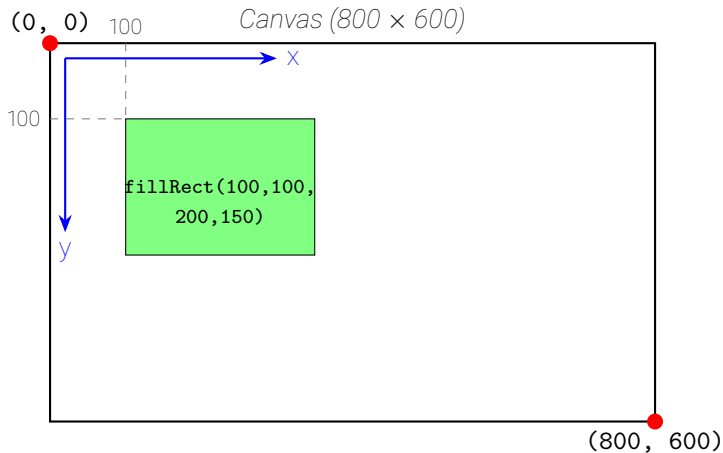
Only **main()** (one line: **launch(args)**) and **start(Stage stage)** (your setup code).

What JavaFX handles

Everything in between: spinning up the UI thread, creating the **Stage**, calling **start()**, and running the event loop that keeps the window alive.

That is why **launch()** lives in **main** and **start()** receives the **Stage** as a parameter -- JavaFX creates the window and hands it to you.

# JavaFX Coordinate System



# Quick Check: Coordinate System

**Without running the code, predict where each shape appears on the canvas ( $800 \times 600$ ):**

1. `gc.fillRect(0, 0, 800, 600)` -- where and how large?
2. `gc.fillOval(300, 200, 100, 100)` -- where on the canvas?
3. `gc.fillRect(0, 0, 400, 300)` -- what portion of the canvas?

# Quick Check: Coordinate System - Answer

(One) Answer)

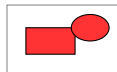
## Answers

1. Fills the **entire canvas** (starts at top-left, full width and height)
2. A circle in the **center-ish area** (starts 300px right, 200px down)
3. The **top-left quarter** of the canvas (half width, half height)

# GraphicsContext: Filled Shapes

`GraphicsContext gc = canvas.getGraphicsContext2D();` gives you the “pen.” For **filled** shapes, set the fill color first:

```
1 gc.setFill(Color.RED);  
2 gc.fillRect(50, 50, 100, 80);  
3 gc.fillOval(200, 50, 100, 100);
```



filled rect & oval

`fillOval(x, y, w, h)` -- circle when `w == h`

# GraphicsContext: Outlined Shapes

For **outlines** (no fill), use `setStroke()` and `setLineWidth()`:

```
1 gc.setStroke(Color.BLUE);  
2 gc.setLineWidth(3);  
3 gc.strokeRect(50, 200, 100, 80);  
4 gc.strokeOval(200, 200, 100, 100);
```



stroked rect & oval



# GraphicsContext: Lines and Text

```
1  gc.strokeLine(400, 50, 500, 200);
2  gc.setFill(Color.BLACK);
3  gc.fillText("Hello!", 400, 250);
```



line & text

- **strokeLine(x1, y1, x2, y2)** -- line from point to point
- **fillText(str, x, y)** -- text at baseline position

# Connecting OOP to JavaFX

Now the key idea: add a **draw()** method to our encapsulated classes:

```
1  class Circle {
2      private double radius;
3      private double x;      // center x position
4      private double y;      // center y position
5      // getters + constructor omitted for brevity
6
7      public void draw(GraphicsContext gc) {
8          double diameter = this.radius * 2;
9          gc.fillOval(this.x - this.radius,
10                     this.y - this.radius,
11                     diameter, diameter);
12     }
13 }
```

Each shape knows **how to draw itself**. The main program just calls `circle.draw(gc)`.

# Your Turn: Write `draw()` for Rectangle

## Mini-Challenge

You just saw how `Circle` draws itself using `gc.fillOval(...)`.

A `Rectangle` has fields: `private double width, height, x, y`; (where `x, y` is the top-left corner).

**Write the `draw(GraphicsContext gc)` method for `Rectangle`.**

*Hint:* The `GraphicsContext` method is `gc.fillRect(x, y, width, height)`.

# Write `draw()` for Rectangle - Solution

Solution

```
public void draw(GraphicsContext gc) {  
    gc.fillRect(this.x, this.y, this.width, this.height);  
}
```

Simpler than Circle -- because Rectangle's **x**, **y** is already the top-left corner!

# Drawing a Rectangle

```
1  class Rectangle {
2      private double width, height, x, y;
3
4      Rectangle(double width, double height, double x, double y) {
5          this.width = Math.max(0, width);
6          this.height = Math.max(0, height);
7          this.x = x;
8          this.y = y;
9      }
10     // getters omitted for brevity
11
12     public void draw(GraphicsContext gc) {
13         gc.fillRect(this.x, this.y, this.width, this.height);
14     }
15 }
```

Notice how the constructor validates inputs (encapsulation!) and the **draw()** method uses the object's own fields.

# Using Custom Classes in JavaFX

Creating objects and rendering them on canvas:

```
@Override
public void start(Stage stage) {
    Canvas canvas = new Canvas(800, 600);
    GraphicsContext gc = canvas.getGraphicsContext2D();
    Circle c1 = new Circle(50, 200, 200);
    Circle c2 = new Circle(30, 400, 300);
    Rectangle r1 = new Rectangle(120, 80, 500, 150);
    gc.setFill(Color.RED);    c1.draw(gc);
    gc.setFill(Color.BLUE);  c2.draw(gc);
    gc.setFill(Color.GREEN); r1.draw(gc);
    Pane pane = new Pane(canvas);
    stage.setScene(new Scene(pane));
    stage.setTitle("OOP Shapes");
    stage.show();
}
```

# Composition in Action: Drawing Class

The **Drawing** class manages and renders multiple shapes:

```
class Drawing {
    private Circle[] circles;
    private int count;
    Drawing(int maxShapes) {
        this.circles = new Circle[maxShapes];
        this.count = 0;
    }
    public void addCircle(Circle c) {
        if (this.count < this.circles.length) {
            this.circles[this.count] = c;
            this.count++;
        }
    }
    public void drawAll(GraphicsContext gc) {
        for (int i = 0; i < this.count; i++)
            this.circles[i].draw(gc);
    }
}
```

# Using the Drawing Class

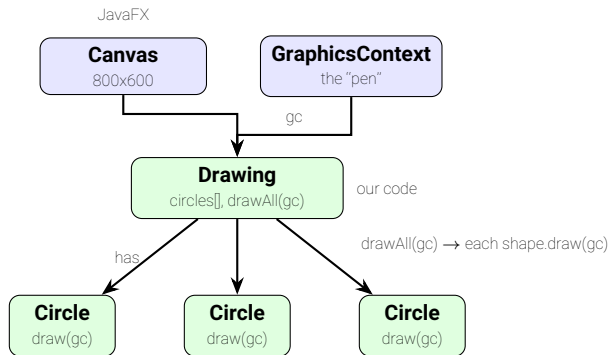
```
1  @Override
2  public void start(Stage stage) {
3      Canvas canvas = new Canvas(800, 600);
4      GraphicsContext gc = canvas.getGraphicsContext2D();
5
6      Drawing drawing = new Drawing(10);
7      drawing.addCircle(new Circle(50, 200, 200));
8      drawing.addCircle(new Circle(30, 400, 300));
9      drawing.addCircle(new Circle(70, 600, 250));
10
11     gc.setFill(Color.CORAL);
12     drawing.drawAll(gc);
13
14     Pane pane = new Pane(canvas);
15     stage.setScene(new Scene(pane));
16     stage.setTitle("Drawing with Composition");
17     stage.show();
18 }
```

## What we achieved

Our **main** code is clean: create a **Drawing**, add shapes, draw all. The details of *how* shapes are drawn are **encapsulated** inside each class.



# How JavaFX, Drawing, and Shapes Relate



**Flow:** `start()` gets Canvas and GC from JavaFX, creates our **Drawing**, adds shapes. `drawing.drawAll(gc)` passes the pen to each shape; each **Circle.draw(gc)** uses it. JavaFX does the pixels; our classes decide *what* to draw.

# Adding Color to Shapes

We can store color as part of the shape (encapsulated, of course):

```
class Circle {
    private double radius, x, y;
    private Color color;
    Circle(double radius, double x, double y, Color color) {
        ... this.color = color;
    }
    public void draw(GraphicsContext gc) {
        gc.setFill(this.color);
        double d = this.radius * 2;
        gc.fillOval(this.x - this.radius, this.y - this.radius, d, d);
    }
}
```

Now each circle manages its own color; the caller no longer needs `gc.setFill()` -- the object handles it.

# Spot the Bug!

```
1  public class MyApp extends Application {
2      @Override
3      public void start(Stage stage) {
4          Canvas canvas = new Canvas(800, 600);
5          GraphicsContext gc = canvas.getGraphicsContext2D();
6
7          Circle[] circles = new Circle[3];
8          circles[0] = new Circle(50, 100, 100);
9          circles[1] = new Circle(30, 200, 200);
10
11          for (int i = 0; i < circles.length; i++) {
12              circles[i].draw(gc); // what happens?
13          }
14      }
15  }
```

# Spot the Bug!

```
1  public class MyApp extends Application {  
2      @Override  
3      public void start(Stage stage) {  
4          Canvas canvas = new Canvas(800, 600);  
5          GraphicsContext gc = canvas.getGraphicsContext2D();  
6  
7          Circle[] circles = new Circle[3];  
8          circles[0] = new Circle(50, 100, 100);  
9          circles[1] = new Circle(30, 200, 200);  
10  
11         for (int i = 0; i < circles.length; i++) {  
12             circles[i].draw(gc); // what happens?  
13         }  
14     }  
15 }
```

# Quick Retrieval: Test Yourself

Before looking at the summary -- can you answer these from memory?

1. What file does Maven use for project configuration?
2. What does **private** do to a field?
3. What is the difference between **==** and **equals()** on objects?
4. Name the five JavaFX components from window to pen.
5. Why do our shape classes have a **draw(GraphicsContext gc)** method?

*Take some time to think, then we go through the answers.*

# Summary

- **Maven** manages dependencies (`pom.xml`) and project structure (`src/main`, `src/test`)
- **Encapsulation**: fields are `private`, accessed through getters/setters with validation
- Prefer **behavior methods** (`takeDamage`) over raw setters (`setHp`)
- `@Override toString()` for readable output, `@Override equals()` for value comparison
- **Composition**: objects containing other objects (has-a relationship)
- **JavaFX**: Stage → Scene → Pane → Canvas → GraphicsContext
- Shape classes encapsulate their data **and** know how to draw themselves